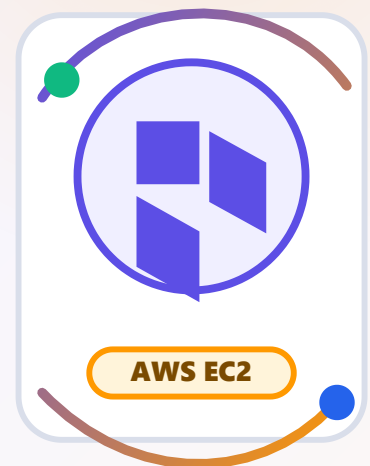




#1 Terraform Day - 1

A beginner-friendly deep handbook for understanding Infrastructure as Code, Terraform fundamentals, AWS setup, and the first EC2 instance project.

IaC Basics | AWS | Terraform CLI | EC2 Mini Project



Prepared by **Satyanarayan Sen**

15+

pages of beginner-focused study material

4

core Terraform workflow commands

1

safe EC2 project walkthrough

0

real credentials or risky secrets included

How To Use This Note

STUDY MAP

This handbook turns Day 1 into a guided study path. Read it once like a story, then return to the diagrams and command cards while practicing.

- | | |
|--|---|
| 1 Infrastructure as Code
Why cloud teams moved from manual clicks to repeatable files. | 2 Why Terraform
Providers, multi-cloud usage, and the workflow mindset. |
| 3 Setup Options
Local install, Codespaces, AWS CLI, and editor basics. | 4 Terraform Vocabulary
Provider, resource, state, plan, module, and variables. |
| 5 HCL Basics
The shape of a simple Terraform file. | 6 AWS Authentication
How Terraform proves who it is before calling AWS APIs. |
| 7 EC2 Mini Project
Planning the first instance without hiding the AWS concepts. | 8 Core Commands
<code>init</code> , <code>plan</code> , <code>apply</code> , and <code>destroy</code> . |
| 9 State And Safety
Why Terraform remembers what it created. | 10 Cheat Sheet
Practice tasks, recap, and trusted sources. |

Learning Promise

By the end, you should be able to explain IaC, read a basic `main.tf`, understand what each Terraform command does, and know why AWS concepts like VPC, subnet, AMI, security group, and key pair matter before running automation.

Cost And Security Reminder

Only practice in an account you control. Never paste real AWS access keys into notes, screenshots, GitHub, or public chats. Destroy lab resources when you finish.

Infrastructure As Code

CONCEPT

Infrastructure as Code, or IaC, means describing cloud infrastructure in files instead of creating it manually from a console.

Manual Infrastructure

- Open AWS Console.
- Click through pages to create VPCs, subnets, EC2, IAM, and security groups.
- Repeat the same steps for every environment.
- Risk: missed settings, undocumented changes, hard rollback.

Infrastructure As Code

- Write desired infrastructure in files.
- Review the change before applying it.
- Reuse the same code for dev, test, and prod with different values.
- Benefit: repeatable, reviewable, version-controlled infrastructure.

From clicks to code

Manual Console Work

Click, remember, repeat
Hard to peer review
Drift appears silently



IaC Workflow

Write files, review, apply
Repeatable across environments
Version history and rollback clues

Repeatability

The same file should create the same intended infrastructure again and again.

Reviewability

Infrastructure changes become pull requests or commits, not hidden console activity.

Collaboration

Teams can discuss infrastructure through code, comments, approvals, and history.

Beginner Translation

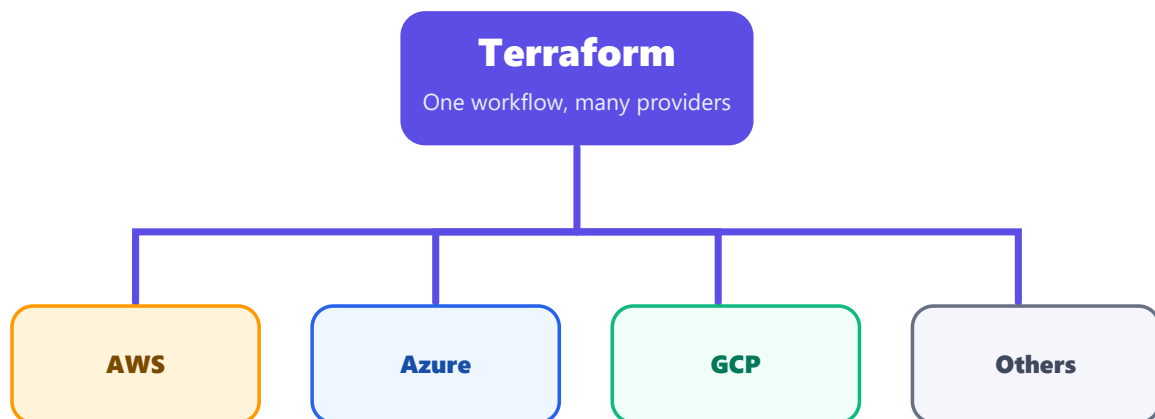
Think of Terraform code as a recipe. The recipe does not cook by itself, but it clearly tells Terraform what ingredients to use and what final dish AWS should prepare.

Why Terraform?

BIG PICTURE

Cloud providers have their own IaC tools, but Terraform became popular because one workflow can manage many platforms through providers.

Tool	Works Best With	Beginner Takeaway
AWS CloudFormation	AWS resources	Powerful inside AWS, but not a universal multi-cloud workflow.
Azure ARM/Bicep	Azure resources	Great for Azure, but tied to that provider ecosystem.
Terraform	AWS, Azure, GCP, Kubernetes, GitHub, and many more	Learn one workflow and use providers to talk to many APIs.



Declarative Style

You describe the desired final state. Terraform figures out what actions are needed to move from the current state to that final state.

Plan Before Apply

`terraform plan` gives a preview. This is one of Terraform's most important safety habits for learners and teams.

Terraform Does Not Replace Cloud Knowledge

Terraform can create AWS resources, but you still need to understand what those resources mean. Day 1 correctly stresses VPC, subnet, AMI, key pair, EC2, and security group basics.

Setup Options

ENVIRONMENT

You can learn Terraform locally or in a cloud development environment. The best setup is the one that lets you practice safely and consistently.

Option A: Local Machine

- Install Terraform CLI.
- Install AWS CLI if you want CLI-based credentials.
- Use VS Code or another editor.
- Store projects in a clean folder and avoid committing secrets.

Option B: GitHub Codespaces

- Use a browser-based dev environment.
- Good for learners who do not want heavy local setup.
- Still protect AWS keys carefully.
- Destroy cloud resources when finished.

1 Install Terraform

Use the official Terraform install page for your operating system. After installation, run `terraform version` to confirm the CLI is available.

2 Prepare AWS Access

Create an IAM user or role with permissions appropriate for a lab. Avoid using root account keys. Prefer short-lived or limited permissions when possible.

3 Configure Credentials

Use `aws configure`, environment variables, or another approved credential method. Never paste actual keys into `main.tf`.

4 Create A Project Folder

Start with a clean directory containing `main.tf`. Terraform stores downloaded provider plugins under `.terraform` after initialization.

Setup Checklist

READY CHECK

Use this page as a final environment check before writing or planning AWS resources with Terraform.

```
# quick checks after setup
terraform version
aws --version
aws sts get-caller-identity
```

What Success Looks Like

You can open a terminal, run `terraform version`, and run an AWS identity check without exposing credentials in files. That is enough to begin the Day 1 EC2 project.

What To Fix First

If Terraform is missing, fix the PATH or installation. If AWS identity fails, fix credentials before touching any Terraform resource code.

Terraform CLI

`terraform version` returns a version.

AWS CLI

`aws --version` works if you choose CLI credentials.

Identity

`aws sts get-caller-identity` shows the expected account.

Folder

Your project folder has a clean `main.tf`.

Secrets

No access keys are written into Terraform files.

Cleanup

You know how to destroy the lab after practice.

Do Not Skip This

Credential and region mistakes create most beginner confusion. Confirm the environment first, then write infrastructure code.

Terraform Vocabulary

Before writing code, learn the words Terraform uses. These terms make the EC2 project easier to understand.

Term	Meaning	Day 1 Example
Provider	A plugin that lets Terraform talk to an external platform API.	<code>provider "aws"</code>
Resource	An infrastructure object Terraform manages.	<code>aws_instance</code> for EC2
Configuration	The <code>.tf</code> files that describe your desired infrastructure.	<code>main.tf</code>
State	Terraform's record of what it manages and the real-world IDs behind it.	<code>terraform.tfstate</code>
Plan	A preview of changes Terraform intends to make.	Create one EC2 instance
Apply	The action that performs the planned changes.	Calls AWS to create EC2
Destroy	The action that removes managed infrastructure.	Terminates the EC2 instance



Provider

The bridge between Terraform and a platform.

Resource

The object you want Terraform to create or manage.

State

The memory Terraform uses to track managed objects.

HCL Structure

SYNTAX

Terraform files use HashiCorp Configuration Language. HCL is designed to be readable, so focus on the block structure first.

```
terraform {  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_instance" "day1_demo" {  
  ami           = "ami-xxxxxxxxxxxxxxxx" # region-specific example only  
  instance_type = "t2.micro"  
  
  tags = {  
    Name = "terraform-day-1-demo"  
  }  
}
```

Block

A block has a type, optional labels, and a body.

Example: `resource "aws_instance"`
`"day1_demo" { ... }`.

Argument

An argument sets a value inside a block.

Example: `instance_type = "t2.micro"`.

Provider Source

`hashicorp/aws` tells Terraform where to download the AWS provider from during `terraform init`.

Tags

Tags are AWS labels. They help you identify cost, owner, project, and purpose later.

HCL Reading Notes

READ FIRST

Once the block structure feels familiar, the next skill is reading configuration safely before running commands.

AMI IDs Are Not Universal

An AMI ID depends on AWS region and date. Do not copy an old AMI blindly. Choose a current AMI from your target region before creating an instance.

Beginner Reading Trick

Read a resource block as a sentence: "Create one AWS instance named day1_demo with this AMI, this instance type, and these tags."

Questions To Ask

- Which provider will this use?
- Which AWS region will receive the request?
- Which resource type is being created?
- Which values are hard-coded placeholders?

Values To Treat Carefully

- Account IDs and ARNs.
- Security group rules.
- Subnet, VPC, and AMI IDs.
- Any value that might reveal a secret.

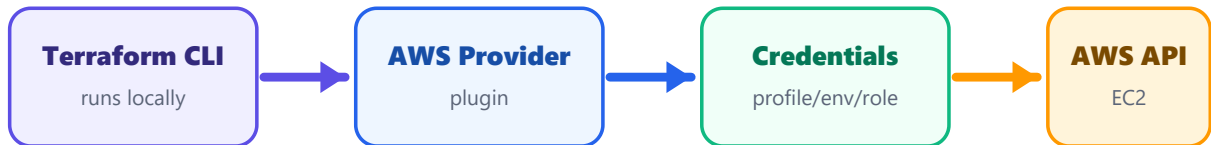
Reading Before Running

Never run a Terraform file just because it is short. Short configuration can still create public resources, open network access, or spend money.

AWS Authentication

SECURITY

Terraform does not magically own your AWS account. It needs credentials, then the AWS provider uses those credentials to call AWS APIs.



Credentials authorize the provider. The provider sends signed requests to AWS.

Safer Patterns

- Use AWS CLI profiles or environment variables.
- Use least privilege for lab users.
- Rotate/delete temporary learning keys after practice.
- Use IAM roles where possible in real teams.

Avoid These Mistakes

- Do not commit access keys.
- Do not hard-code keys in `main.tf`.
- Do not use root account access keys.
- Do not share screenshots that reveal credentials.

```
# preferred beginner check
aws configure
aws sts get-caller-identity

# Terraform can then use the configured AWS credential chain.
```

Secret Rule

If a value can log in, create infrastructure, delete resources, or spend money, treat it as a secret. Never include it in this note, GitHub, screenshots, or messages.

EC2 Mini Project

PROJECT

The first project launches an EC2 instance. The Terraform syntax is short, but the AWS choices behind it matter.

Inputs You Must Know

- **Region:** where the instance will run.
- **AMI:** the operating system image.
- **Instance type:** CPU/memory size such as `t2.micro`.
- **Subnet:** the network location inside a VPC.
- **Key pair:** SSH access option for Linux instances.
- **Security group:** firewall rules for inbound/outbound traffic.

Beginner-Safe Defaults

- Use a free-tier eligible instance type if your account qualifies.
- Use a non-production AWS account or sandbox.
- Tag every resource with project and owner.
- Run `terraform destroy` after practice.
- Review the plan before applying anything.

AWS Region

VPC

Subnet



EC2 Instance

AMI + type + key

Security Group

What Terraform Is Doing

Terraform sends a request to AWS: create an EC2 instance with the settings in the resource block. AWS creates the real object and returns identifiers. Terraform stores those identifiers in state.

Project Code Walkthrough

This sample is intentionally conservative. Replace placeholders with current values from your own AWS region and account before any real lab run.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "day1_ec2" {
  ami          = "ami-xxxxxxxxxxxxxxxx" # choose current AMI for your region
  instance_type = "t2.micro"
  subnet_id    = "subnet-xxxxxxxxxxxxxxxx"
  key_name     = "your-existing-key-pair-name"

  tags = {
    Name     = "terraform-day-1"
    Owner    = "Satyanarayan Sen"
    ManagedBy = "Terraform"
  }
}
```

Line/Block	Purpose	Beginner Check
terraform	Defines required provider plugins.	Did terraform init download the provider?
provider "aws"	Selects AWS and region.	Is the region where your AMI and subnet exist?
aws_instance	Declares the EC2 instance.	Do all IDs match your account and region?
tags	Adds labels to the AWS resource.	Can you recognize this lab resource later?

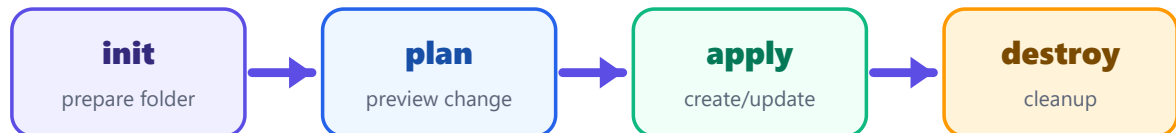
Do Not Paste This Blindly

The AMI, subnet, and key pair values are placeholders. A wrong region or missing key pair can cause errors, and an open security group can create risk. Learn the AWS concepts before applying.

The Terraform Lifecycle

WORKFLOW

Day 1 centers on four commands. Memorize the order and the purpose, not just the spelling.



Plan before apply. Destroy when the lab is finished.

terraform init

01

Initializes the working directory. Downloads providers and prepares backend-related files.

safe

first run

plugins

terraform plan

02

Builds an execution plan. Shows what Terraform would create, change, or destroy.

review

dry run

no cloud change

terraform apply

03

Applies the planned actions after confirmation. This can create billable AWS resources.

changes AWS

confirm

terraform destroy

04

Removes resources managed by this configuration and state. Use it to clean up labs.

cleanup

cost control

Running The First Workflow

TERMINAL

A clean workflow gives you confidence. The command order matters because Terraform needs setup, review, action, and cleanup.

```
# 1. Enter the project folder
cd terraform-day-1

# 2. Initialize providers
terraform init

# 3. Preview the changes
terraform plan

# 4. Apply only after reviewing the plan
terraform apply

# 5. Clean up lab resources when finished
terraform destroy
```

When Plan Looks Good

The plan should say Terraform will create the expected resource. For Day 1, that usually means one EC2 instance and any related resources you explicitly defined.

When Plan Looks Wrong

Stop. Fix the configuration, region, credentials, or IDs first. The whole point of `plan` is to catch surprises before AWS changes.

Command	Cloud Resources Changed?	Typical Beginner Emotion	Best Habit
<code>init</code>	No	"Did it install?"	Read provider download messages.
<code>plan</code>	No	"What will happen?"	Read every create/update/delete line.
<code>apply</code>	Yes	"Now it is real."	Confirm only after review.
<code>destroy</code>	Yes	"Clean up costs."	Verify the resource is gone in AWS.

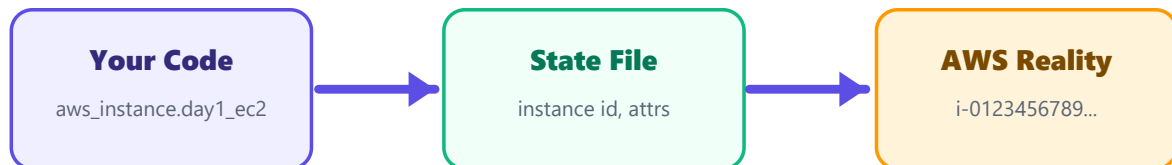
Apply And Destroy Are Real Actions

This note documents the commands, but it does not run them. Use a controlled lab account and verify every plan before confirming.

Understanding State

STATE

State is Terraform's memory. It maps your code to real cloud resources and helps Terraform know what changed.



Terraform compares configuration + state + provider data to decide the next plan.

Why State Matters

- Tracks resource IDs.
- Helps Terraform detect drift.
- Connects local names to cloud objects.
- Supports planning future changes.

Beginner State Safety

- Do not manually edit state files.
- Do not commit state if it may contain sensitive data.
- For teams, use a remote backend with locking.
- Keep Day 1 labs simple and isolated.

Simple Mental Model

Your code says what you want. AWS has what really exists. State is Terraform's notebook that connects the two.

Common Day 1 Errors

TROUBLESHOOT

Most beginner errors are not mysterious. They usually come from setup, credentials, region mismatch, missing AWS objects, or old AMI values.

Symptom	Likely Cause	What To Check
<code>terraform</code> not recognized	CLI not installed or PATH not updated.	Restart terminal, verify install, run <code>terraform version</code> .
Provider download fails	Network, proxy, or provider source issue.	Check internet, retry <code>terraform init</code> , verify provider block.
Invalid AMI ID	AMI does not exist in selected region.	Pick a current AMI from the same AWS region.
Subnet not found	Subnet ID from another region or account.	Confirm AWS account, region, and VPC/subnet details.
Key pair not found	<code>key_name</code> does not match existing key pair.	Check EC2 key pairs in that region.
Access denied	Credentials lack permission.	Inspect IAM policy or use a suitable lab role/user.

Region

Does every ID belong to the same AWS region?

Account

Are your credentials pointing to the account you expect?

AMI

Is the image current and valid in that region?

Subnet

Does the subnet exist and allow your intended use?

Key Pair

Does the name exactly match an EC2 key pair?

Cleanup

Have you destroyed resources after finishing?

Debug In This Order

Check credentials first, then region, then resource IDs, then Terraform syntax. This order solves many Day 1 problems quickly.

Cleanup And Cost Control

SAFETY

Learning cloud tools is fun until forgotten resources generate charges. Day 1 should end with cleanup discipline.

Always Destroy Lab Resources

After practice, run `terraform destroy` from the same project folder and confirm the result in the AWS Console. Deleted infrastructure cannot serve traffic, but it also stops avoidable lab spending.

```
# cleanup workflow
terraform plan -destroy
terraform destroy

# optional verification ideas
aws ec2 describe-instances --filters "Name=tag:Name,Values=terraform-day-1"
aws sts get-caller-identity
```

Before Destroy

- Confirm this is only a lab project.
- Check you are in the correct folder.
- Check you are using the intended AWS account.
- Read the destroy plan.

After Destroy

- Refresh the EC2 console.
- Look for leftover resources.
- Check billing if you experimented beyond EC2.
- Remove temporary access keys if no longer needed.



Practice Tasks

HOMEWORK

Use these tasks to convert the video into muscle memory. Do the reading tasks even if you are not ready to run AWS resources.

1 Explain IaC In Your Own Words

Write three sentences comparing manual cloud work with Terraform-managed infrastructure.

2 Read A `main.tf`

Identify provider, resource type, resource name, arguments, and tags without running anything.

3 Install Or Open A Practice Environment

Confirm `terraform version`. If using Codespaces, understand where files and credentials live.

4 Find AWS Values

In the AWS Console, locate your region, one valid AMI, a subnet, a key pair, and an appropriate security group conceptually.

5 Run Safe Commands First

Practice `terraform fmt`, `terraform validate`, `terraform init`, and `terraform plan` before applying.

6 Document Cleanup

Write down exactly how you will destroy lab resources before creating them.

Beginner Milestone

You are ready for Day 2 when you can look at a Terraform resource block and predict what cloud object it is trying to create.

Day 1 Cheat Sheet

RECAP

Keep this page nearby while practicing. It is the fast recall version of the whole note.

init

setup

Initializes a working directory and downloads provider plugins.

```
terraform init
```

plan

preview

Shows what Terraform will do before it changes infrastructure.

```
terraform plan
```

apply

change

Creates, updates, or deletes resources after confirmation.

```
terraform apply
```

destroy

cleanup

Removes resources managed by the current configuration and state.

```
terraform destroy
```

Remember

- Terraform is declarative.
- Providers connect Terraform to platforms.
- State maps code to real infrastructure.
- Plans should be reviewed carefully.

Never Forget

- No hard-coded AWS keys.
- No blind apply.
- No old AMI copy-paste.
- No forgotten lab resources.

Core Takeaway

Terraform syntax matters, but cloud understanding matters just as much. Know what AWS resource you are asking Terraform to create.

Sources And References

REFERENCES

These sources informed the Day 1 note and should be used for official command behavior, installation steps, and AWS credential details.

- Abhishek Veeramalla, Terraform Zero to Hero Day 1 video: youtube.com/watch?v=fgp-t5SqQmM
- Day 1 GitHub materials: github.com/iam-veeramalla/terraform-zero-to-hero/tree/main/Day-1
- Terraform overview: developer.hashicorp.com/terraform/intro
- Terraform install guide: developer.hashicorp.com/terraform/install
- Terraform `init` command: developer.hashicorp.com/terraform/cli/commands/init
- Terraform `plan` command: developer.hashicorp.com/terraform/cli/commands/plan
- Terraform `apply` command: developer.hashicorp.com/terraform/cli/commands/apply
- Terraform `destroy` command: developer.hashicorp.com/terraform/cli/commands/destroy
- AWS CLI configuration files: docs.aws.amazon.com/cli/latest/userguide/cli-configure-files.html

Prepared For Study Use

This document is a learning note, not production cloud architecture guidance. Always verify current provider versions, AMIs, AWS pricing, and security requirements before using Terraform in real environments.

IaC

Write infrastructure as
versioned code

Plan

Preview before changes

State

Track managed resources

Clean

Destroy labs after
practice